

Robot Arm Control System

Generated by Doxygen 1.9.8

1 TÍMOVÝ PROJEKT	1
1.1 Vnorený radiaci systém pre model robotického manipulátora	1
1.1.1 Riešitelia	1
1.1.2 Vedúci projektu	1
1.1.3 Branches	1
1.1.4 Dokumentácia	1
2 Namespace Index	3
2.1 Namespace List	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Namespace Documentation	11
6.1 Robot Namespace Reference	11
6.1.1 Detailed Description	11
7 Class Documentation	13
7.1 actuator_parameters Struct Reference	13
7.1.1 Detailed Description	13
7.2 Robot::JointActuators Class Reference	13
7.2.1 Detailed Description	14
7.2.2 Member Function Documentation	14
7.2.2.1 writeAngles()	14
7.3 Robot::RobotArm< DOF > Class Template Reference	14
7.3.1 Detailed Description	15
7.3.2 Constructor & Destructor Documentation	16
7.3.2.1 RobotArm()	16
7.3.3 Member Function Documentation	16
7.3.3.1 getCurrentEffectorPositionOrientation()	16
7.3.3.2 getJointAngle()	16
7.3.3.3 getJointAngles()	16
7.3.3.4 goTo()	17
7.3.3.5 setJointAngles()	17
7.4 Robot::RobotKinematics< DOF > Class Template Reference	17
7.4.1 Detailed Description	19
7.4.2 Constructor & Destructor Documentation	19
7.4.2.1 RobotKinematics()	19

7.4.3 Member Function Documentation	19
7.4.3.1 forwardKinematics()	19
7.4.3.2 getJacobian()	20
7.4.3.3 getRotDerivatives()	20
7.4.3.4 SolveIK()	20
7.5 Robot::RobotKinematicsBase< DOF > Class Template Reference	21
7.5.1 Detailed Description	22
7.5.2 Member Function Documentation	22
7.5.2.1 forwardKinematics()	22
7.5.2.2 SolveIK()	22
7.6 RobotServoController Class Reference	23
7.6.1 Detailed Description	24
7.6.2 Constructor & Destructor Documentation	24
7.6.2.1 RobotServoController()	24
7.6.3 Member Function Documentation	25
7.6.3.1 init()	25
7.6.3.2 writeAngles()	25
7.7 RotationMatrix Class Reference	25
7.7.1 Detailed Description	26
7.7.2 Member Function Documentation	26
7.7.2.1 d()	26
7.7.2.2 operator>() [1/2]	27
7.7.2.3 operator>() [2/2]	27
7.8 RotationX Class Reference	27
7.8.1 Detailed Description	28
7.8.2 Member Function Documentation	29
7.8.2.1 d()	29
7.8.2.2 operator>() [1/2]	29
7.8.2.3 operator>() [2/2]	29
7.9 RotationY Class Reference	29
7.9.1 Detailed Description	30
7.9.2 Member Function Documentation	31
7.9.2.1 d()	31
7.9.2.2 operator>() [1/2]	31
7.9.2.3 operator>() [2/2]	31
7.10 RotationZ Class Reference	31
7.10.1 Detailed Description	32
7.10.2 Member Function Documentation	33
7.10.2.1 d()	33
7.10.2.2 operator>() [1/2]	33
7.10.2.3 operator>() [2/2]	33

8 File Documentation	35
8.1 TP26-Roboticke-rameno/geometry.h File Reference	35
8.1.1 Detailed Description	37
8.1.2 Function Documentation	37
8.1.2.1 drotX() [1/2]	37
8.1.2.2 drotX() [2/2]	37
8.1.2.3 drotY() [1/2]	38
8.1.2.4 drotY() [2/2]	38
8.1.2.5 drotZ() [1/2]	38
8.1.2.6 drotZ() [2/2]	39
8.1.2.7 rotX() [1/2]	39
8.1.2.8 rotX() [2/2]	39
8.1.2.9 rotY() [1/2]	39
8.1.2.10 rotY() [2/2]	40
8.1.2.11 rotZ() [1/2]	40
8.1.2.12 rotZ() [2/2]	40
8.2 geometry.h	41
8.3 TP26-Roboticke-rameno/robot.h File Reference	43
8.3.1 Detailed Description	44
8.4 robot.h	44
8.5 TP26-Roboticke-rameno/ServoActuator.h File Reference	47
8.5.1 Detailed Description	48
8.6 ServoActuator.h	48
8.7 TP26-Roboticke-rameno/STM32FreeRTOSConfig.h File Reference	49
8.7.1 Detailed Description	50
8.8 STM32FreeRTOSConfig.h	50
8.9 TP26-Roboticke-rameno/ui_screens.h File Reference	52
8.9.1 Detailed Description	53
8.9.2 Function Documentation	53
8.9.2.1 drawRPY()	53
8.9.2.2 drawXYZ()	54
8.10 ui_screens.h	54
Index	57

Chapter 1

TÍMOVÝ PROJEKT

1.1 Vnorený riadiaci systém pre model robotického manipulátora

placeholder text - opis projektu

1.1.1 Riešitelia

- Bc. Simona Walterová - Team Leader
- Bc. Samuel Vavruš - Tech Leader
- Bc. Samuel Valko
- Bc. Samuel Kopp
- Bc. Márk Pšenák
- Bc. Filip Dubovský
- Bc. Jakub Janega

1.1.2 Vedúci projektu

- Ing. Martin Dodek, PhD.
-

1.1.3 Branches

- `main` - default branch
 - `devel` - branch pre vývoj projektu
 - `docs` - branch pre vývoj dokumentácie
-

1.1.4 Dokumentácia

- dokumentácia k zadaniu - [thesis.pdf](#)
- Doxygen dokumentácia ku zdrojovému kódu - TU TREBA NASTAVIT GITHUB PAGES ABY FUNGOVAL PRE-LINK NA DOXYGEN [github.com/samovavrus/TP26-Roboticke-rameno/.../index.html](https://github.com/samovavrus/TP26-Roboticke-rameno/blob/master/docs/index.html)

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

Robot	Core robotics control and kinematics namespace	11
-----------------------	--	----

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

actuator_parameters	13
Robot::JointActuators	13
RobotServoController	23
Robot::RobotArm< DOF >	14
Robot::RobotKinematicsBase< DOF >	21
Robot::RobotKinematics< DOF >	17
RotationMatrix	25
RotationX	27
RotationY	29
RotationZ	31

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

actuator_parameters	Joint actuator configuration and calibration parameters	13
Robot::JointActuators	Abstract base class for joint actuation hardware	13
Robot::RobotArm< DOF >	High-level robot arm control interface	14
Robot::RobotKinematics< DOF >	Concrete kinematics solver using Jacobian-based inverse kinematics	17
Robot::RobotKinematicsBase< DOF >	Abstract base class for robot kinematics solvers	21
RobotServoController	Hardware interface for servo motor control via PCA9685 PWM driver	23
RotationMatrix	Abstract base class for joint rotation matrix generation	25
RotationX	Rotation in X-axis (right-hand rule: thumb along +X)	27
RotationY	Rotation in Y-axis (right-hand rule: thumb along +Y)	29
RotationZ	Rotation in Z-axis (right-hand rule: thumb along +Z)	31

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

TP26-Roboticke-rameno/ geometry.h	
Rotation matrices and derivatives for robot kinematics	35
TP26-Roboticke-rameno/ robot.h	
Robot kinematics and actuator interfaces	43
TP26-Roboticke-rameno/ ServoActuator.h	
Servo actuator controller using PCA9685	47
TP26-Roboticke-rameno/ STM32FreeRTOSConfig.h	
FreeRTOS configuration for STM32 targets	49
TP26-Roboticke-rameno/ ui_screens.h	
User interface display functions for LCD output	52

Chapter 6

Namespace Documentation

6.1 Robot Namespace Reference

Core robotics control and kinematics namespace.

Classes

- class [JointActuators](#)
Abstract base class for joint actuation hardware.
- class [RobotArm](#)
High-level robot arm control interface.
- class [RobotKinematics](#)
Concrete kinematics solver using Jacobian-based inverse kinematics.
- class [RobotKinematicsBase](#)
Abstract base class for robot kinematics solvers.

6.1.1 Detailed Description

Core robotics control and kinematics namespace.

Contains base classes and implementations for robot arm kinematic control, including forward kinematics, inverse kinematics (IK), and joint actuation.

Chapter 7

Class Documentation

7.1 `actuator_parameters` Struct Reference

Joint actuator configuration and calibration parameters.

```
#include <robot.h>
```

Public Attributes

- float **minAngle**
Minimum joint angle in radians.
- float **maxAngle**
Maximum joint angle in radians.
- float **gain**
Scaling factor for angle-to-PWM conversion.
- float **offset**
Offset applied to joint angles during conversion.

7.1.1 Detailed Description

Joint actuator configuration and calibration parameters.

Stores calibration and limit parameters for a single joint actuator. Used to convert between joint angle space and PWM control signals.

The documentation for this struct was generated from the following file:

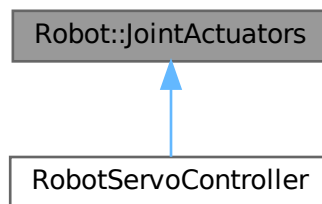
- TP26-Roboticke-rameno/[robot.h](#)

7.2 `Robot::JointActuators` Class Reference

Abstract base class for joint actuation hardware.

```
#include <robot.h>
```

Inheritance diagram for Robot::JointActuators:



Public Member Functions

- **JointActuators** ()
Default constructor.
- **virtual bool writeAngles** (const float *, std::size_t)=0
Sends joint angle commands to all actuators.

7.2.1 Detailed Description

Abstract base class for joint actuation hardware.

Defines the interface for sending computed joint angles to physical actuators (servos, motors, etc.). Implementations handle conversion from angles to hardware control signals (PWM, current commands, etc.).

7.2.2 Member Function Documentation

7.2.2.1 writeAngles()

```
virtual bool Robot::JointActuators::writeAngles (
    const float * ,
    std::size_t ) [pure virtual]
```

Sends joint angle commands to all actuators.

Parameters

<i>angles</i>	Array of joint angles in radians
<i>n</i>	Number of joints (should match DOF)

Returns

true if all angles were within valid ranges and written successfully, false if any angle exceeded joint limits

Implemented in [RobotServoController](#).

The documentation for this class was generated from the following file:

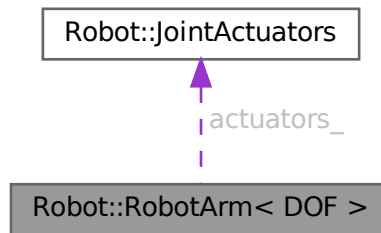
- TP26-Roboticke-rameno/[robot.h](#)

7.3 Robot::RobotArm< DOF > Class Template Reference

High-level robot arm control interface.

```
#include <robot.h>
```

Collaboration diagram for Robot::RobotArm< DOF >:



Public Member Functions

- [RobotArm](#) ([RobotKinematicsBase](#)< [DOF](#) > *[robot_kinematics](#), [JointActuators](#) *[actuators](#))
Initializes robot arm with kinematics solver and actuator hardware.
- [~RobotArm](#) ()=[default](#)
Destructor.
- [bool](#) [setJointAngles](#) ([const](#) [Matrix](#)< [DOF](#), 1 > &[angles](#))
Sets and commits joint angles to hardware actuators.
- [float](#) [getJointAngle](#) ([std::size_t](#) [index](#)) [const](#)
Gets current angle of a single joint.
- [Matrix](#)< [DOF](#), 1 > [getJointAngles](#) () [const](#)
Gets current angles of all joints.
- [std::pair](#)< [Matrix](#)< 3, 1 >, [Matrix](#)< 3, 3 > > [getCurrentEffectorPositionOrientation](#) () [const](#)
Computes current end-effector pose from joint state.
- [bool](#) [goTo](#) ([const](#) [Matrix](#)< 3, 1 > &[pr](#), [const](#) [Matrix](#)< 3, 3 > &[Rr](#))
Moves robot end-effector to desired position and orientation.

Protected Attributes

- [JointActuators](#) * [actuators_](#)
Pointer to joint actuator hardware interface.
- [Matrix](#)< [DOF](#), 1 > [jointAngles_](#)
Current joint angles in radians.
- [RobotKinematicsBase](#)< [DOF](#) > * [robot_kinematics_](#)
Pointer to kinematics solver.

7.3.1 Detailed Description

```
template<std::size_t DOF>
class Robot::RobotArm< DOF >
```

High-level robot arm control interface.

Template Parameters

<i>DOF</i>	Degrees of freedom (number of joints)
------------	---------------------------------------

Coordinates kinematics solvers with hardware actuators to provide intuitive end-effector-centric control. Maintains

current joint state and manages both forward kinematics (state monitoring) and inverse kinematics (motion planning).

7.3.2 Constructor & Destructor Documentation

7.3.2.1 RobotArm()

```
template<std::size_t DOF>
Robot::RobotArm< DOF >::RobotArm (
    RobotKinematicsBase< DOF > * robot_kinematics,
    JointActuators * actuators ) [inline]
```

Initializes robot arm with kinematics solver and actuator hardware.

Parameters

<i>robot_kinematics</i>	Pointer to kinematics solver instance
<i>actuators</i>	Pointer to joint actuators instance

Does not take ownership of pointers; caller must manage object lifetimes.

7.3.3 Member Function Documentation

7.3.3.1 getCurrentEffectorPositionOrientation()

```
template<std::size_t DOF>
std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > Robot::RobotArm< DOF >::getCurrentEffector↔
PositionOrientation ( ) const [inline]
```

Computes current end-effector pose from joint state.

Returns

Pair of (end-effector position [x,y,z] in meters, end-effector rotation matrix 3x3)

7.3.3.2 getJointAngle()

```
template<std::size_t DOF>
float Robot::RobotArm< DOF >::getJointAngle (
    std::size_t index ) const [inline]
```

Gets current angle of a single joint.

Parameters

<i>index</i>	Joint index (0 to DOF-1)
--------------	--------------------------

Returns

Joint angle in radians

7.3.3.3 getJointAngles()

```
template<std::size_t DOF>
Matrix< DOF, 1 > Robot::RobotArm< DOF >::getJointAngles ( ) const [inline]
```

Gets current angles of all joints.

Returns

Vector of DOF joint angles in radians

7.3.3.4 goTo()

```
template<std::size_t DOF>
bool Robot::RobotArm< DOF >::goTo (
    const Matrix< 3, 1 > & pr,
    const Matrix< 3, 3 > & Rr ) [inline]
```

Moves robot end-effector to desired position and orientation.

Parameters

<i>pr</i>	Desired end-effector position [x, y, z] in meters
<i>Rr</i>	Desired end-effector rotation matrix (3x3)

Returns

true if IK solution found and successfully moved to target, false if IK failed or joint angles exceeded limits

Solves inverse kinematics and updates joint actuators if solution found. Uses predefined convergence tolerances (1e-5 m position, 1e-3 rad orientation) and Levenberg-Marquardt damping factor 1e-2 with maximum 50 iterations.

7.3.3.5 setJointAngles()

```
template<std::size_t DOF>
bool Robot::RobotArm< DOF >::setJointAngles (
    const Matrix< DOF, 1 > & angles ) [inline]
```

Sets and commits joint angles to hardware actuators.

Parameters

<i>angles</i>	Target joint angles in radians
---------------	--------------------------------

Returns

true if angles were within limits and successfully written, false if any angle exceeded joint constraints

Updates internal state only if actuators successfully write the angles.

The documentation for this class was generated from the following file:

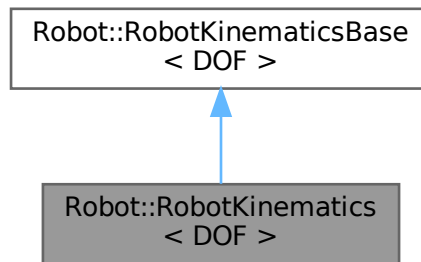
- TP26-Roboticke-rameno/[robot.h](#)

7.4 Robot::RobotKinematics< DOF > Class Template Reference

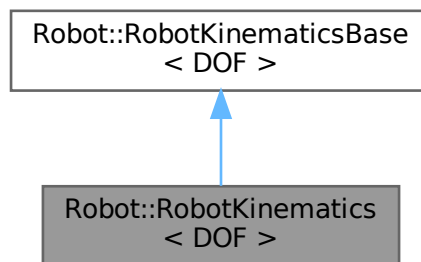
Concrete kinematics solver using Jacobian-based inverse kinematics.

```
#include <robot.h>
```

Inheritance diagram for Robot::RobotKinematics< DOF >:



Collaboration diagram for Robot::RobotKinematics< DOF >:



Public Member Functions

- [RobotKinematics](#) ([const](#) std::array< [Matrix](#)< 3, 1 >, [DOF](#) > &[L_](#), [const](#) std::array< [const](#) [RotationMatrix](#) *, [DOF](#) > &[R_](#))
Initializes kinematics solver with robot link parameters.
- std::pair< [Matrix](#)< 3, 1 >, [Matrix](#)< 3, 3 > > [forwardKinematics](#) ([const](#) [Matrix](#)< [DOF](#), 1 > &[theta](#)) [const](#) [override](#)
Computes end-effector position and orientation from joint angles.
- std::array< [Matrix](#)< 3, 3 >, [DOF](#) > [getRotDerivatives](#) ([const](#) [Matrix](#)< [DOF](#), 1 > &[theta](#)) [const](#)
Computes partial derivatives of rotation matrices with respect to joint angles.
- [Matrix](#)< 3, [DOF](#) > [getJacobian](#) ([const](#) [Matrix](#)< [DOF](#), 1 > &[theta](#)) [const](#)
Computes 3×DOF geometric Jacobian for end-effector linear velocity.
- [bool](#) [SolveIK](#) ([const](#) [Matrix](#)< 3, 1 > &[pr](#), [const](#) [Matrix](#)< 3, 3 > &[Rr](#), [Matrix](#)< [DOF](#), 1 > &[theta](#), [float](#) [tol_pos](#), [float](#) [tol_ori](#), [float](#) [lambda](#), [int](#) [max_iter](#)) [override](#)
Solves inverse kinematics using Levenberg-Marquardt optimization.

7.4.1 Detailed Description

```
template<std::size_t DOF>
class Robot::RobotKinematics< DOF >
```

Concrete kinematics solver using Jacobian-based inverse kinematics.

Template Parameters

<i>DOF</i>	Degrees of freedom (number of joints)
------------	---------------------------------------

Implements forward and inverse kinematics for a robot arm using:

- Forward kinematics: Composition of rotation matrices and translations
- Inverse kinematics: Levenberg-Marquardt numerical optimization with analytical Jacobian computation for both position and orientation errors

7.4.2 Constructor & Destructor Documentation

7.4.2.1 RobotKinematics()

```
template<std::size_t DOF>
Robot::RobotKinematics< DOF >::RobotKinematics (
    const std::array< Matrix< 3, 1 >, DOF > & L_,
    const std::array< const RotationMatrix *, DOF > & R_ ) [inline]
```

Initializes kinematics solver with robot link parameters.

Parameters

<i>L_</i>	Array of link translation vectors (DH parameter d)
<i>R_</i>	Array of rotation function pointers for each joint Each RotationMatrix* implements R(theta) = rotation matrix for that joint

7.4.3 Member Function Documentation

7.4.3.1 forwardKinematics()

```
template<std::size_t DOF>
std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > Robot::RobotKinematics< DOF >::forwardKinematics (
    const Matrix< DOF, 1 > & theta ) const [inline], [override], [virtual]
```

Computes end-effector position and orientation from joint angles.

Parameters

<i>theta</i>	Joint angles in radians
--------------	-------------------------

Returns

Pair of (end-effector position [x,y,z], end-effector rotation matrix)

Implements DH-based forward kinematics: $p = (R_0 * R_1 * \dots * R_n) * (L_0 + L_1 + \dots)$

Implements [Robot::RobotKinematicsBase< DOF >](#).

7.4.3.2 getJacobian()

```
template<std::size_t DOF>
Matrix< 3, DOF > Robot::RobotKinematics< DOF >::getJacobian (
    const Matrix< DOF, 1 > & theta ) const [inline]
```

Computes 3×DOF geometric Jacobian for end-effector linear velocity.

Parameters

<i>theta</i>	Current joint angles in radians
--------------	---------------------------------

Returns

3×DOF Jacobian matrix: $dp/dtheta$

Relates joint angular velocities to end-effector linear velocity: $v_end = J * dtheta/dt$
Each column j represents the contribution of joint j 's rotation to end-effector velocity.

7.4.3.3 getRotDerivatives()

```
template<std::size_t DOF>
std::array< Matrix< 3, 3 >, DOF > Robot::RobotKinematics< DOF >::getRotDerivatives (
    const Matrix< DOF, 1 > & theta ) const [inline]
```

Computes partial derivatives of rotation matrices with respect to joint angles.

Parameters

<i>theta</i>	Current joint angles in radians
--------------	---------------------------------

Returns

Array of DOF matrices: $dR_i/dtheta_i$ for each joint i

Used internally by inverse kinematics to build the Jacobian for orientation error (last 3 rows). Computed using: $dE_i = dR_i/dtheta_i * R^T$ where dR_i is the derivative of i -th rotation matrix

7.4.3.4 SolveIK()

```
template<std::size_t DOF>
bool Robot::RobotKinematics< DOF >::SolveIK (
    const Matrix< 3, 1 > & pr,
    const Matrix< 3, 3 > & Rr,
    Matrix< DOF, 1 > & theta,
    float tol_pos,
    float tol_ori,
    float lambda,
    int max_iter ) [inline], [override], [virtual]
```

Solves inverse kinematics using Levenberg-Marquardt optimization.

Parameters

<i>pr</i>	Desired end-effector position [x, y, z] in meters
<i>Rr</i>	Desired end-effector rotation matrix (3×3)

Parameters

<i>theta</i>	[in/out] Starting joint angles (input), IK solution (output)
<i>tol_pos</i>	Position error tolerance in meters
<i>tol_ori</i>	Orientation error tolerance in radians (extracted from R_error)
<i>lambda</i>	Damping factor for Levenberg-Marquardt regularization
<i>max_iter</i>	Maximum iterations before giving up

Returns

true if converged to solution meeting both tolerances, false if max iterations exceeded without convergence

Algorithm:

1. Compute forward kinematics for current pose
2. Calculate position error: $rp = p_{\text{current}} - p_{\text{desired}}$
3. Calculate orientation error from rotation matrix: $r0 = [R_err(1,0), R_err(2,0), R_err(2,1)]$
4. Build 6×DOF Jacobian combining position and orientation derivatives
5. Apply Levenberg-Marquardt damping to Jacobian diagonal
6. Solve for joint angle updates: $\Delta_{\text{theta}} = J^{-1} * (-\text{residuals})$
7. Update joint angles and repeat until convergence

Implements [Robot::RobotKinematicsBase< DOF >](#).

The documentation for this class was generated from the following file:

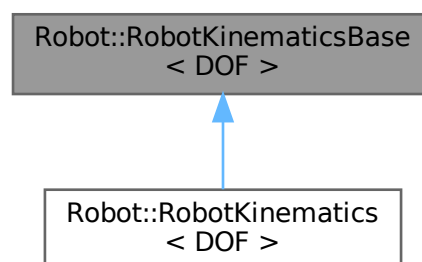
- TP26-Roboticke-rameno/[robot.h](#)

7.5 Robot::RobotKinematicsBase< DOF > Class Template Reference

Abstract base class for robot kinematics solvers.

```
#include <robot.h>
```

Inheritance diagram for Robot::RobotKinematicsBase< DOF >:



Public Member Functions

- `virtual bool SolveIK (const Matrix< 3, 1 > &pr, const Matrix< 3, 3 > &Rr, Matrix< DOF, 1 > &theta_, float tol_pos, float tol_ori, float lambda, int max_iter)=0`
Solves inverse kinematics using numerical optimization.
- `virtual std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > forwardKinematics (const Matrix< DOF, 1 > &theta) const =0`
Computes end-effector position and orientation from joint angles.

7.5.1 Detailed Description

`template<std::size_t DOF>`
class Robot::RobotKinematicsBase< DOF >

Abstract base class for robot kinematics solvers.

Template Parameters

<i>DOF</i>	Degrees of freedom (number of joints)
------------	---------------------------------------

Defines the interface for forward and inverse kinematics computations. Concrete implementations must solve both FK (joint angles → end-effector pose) and IK (desired pose → joint angles).

7.5.2 Member Function Documentation

7.5.2.1 forwardKinematics()

```
template<std::size_t DOF>
virtual std::pair< Matrix< 3, 1 >, Matrix< 3, 3 > > Robot::RobotKinematicsBase< DOF >::forwardKinematics (
    const Matrix< DOF, 1 > & theta ) const [pure virtual]
```

Computes end-effector position and orientation from joint angles.

Parameters

<i>theta</i>	Joint angles in radians
--------------	-------------------------

Returns

Pair of (position vector [x,y,z], rotation matrix 3x3)

Implemented in `Robot::RobotKinematics< DOF >`.

7.5.2.2 SolveIK()

```
template<std::size_t DOF>
virtual bool Robot::RobotKinematicsBase< DOF >::SolveIK (
    const Matrix< 3, 1 > & pr,
    const Matrix< 3, 3 > & Rr,
    Matrix< DOF, 1 > & theta_,
    float tol_pos,
    float tol_ori,
    float lambda,
    int max_iter ) [pure virtual]
```

Solves inverse kinematics using numerical optimization.

Parameters

<i>pr</i>	Desired end-effector position [x, y, z] in meters
-----------	---

Parameters

<i>Rr</i>	Desired end-effector rotation matrix (3x3)
<i>theta_</i>	[in/out] Joint angles: initialized with starting values, updated with IK solution
<i>tol_pos</i>	Position tolerance in meters (convergence criterion)
<i>tol_ori</i>	Orientation tolerance in radians (convergence criterion)
<i>lambda</i>	Damping factor for Levenberg-Marquardt regularization
<i>max_iter</i>	Maximum number of iterations

Returns

true if solution found within tolerance, false otherwise

Implemented in [Robot::RobotKinematics< DOF >](#).

The documentation for this class was generated from the following file:

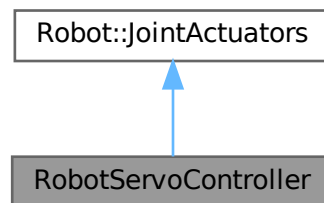
- TP26-Roboticke-rameno/[robot.h](#)

7.6 RobotServoController Class Reference

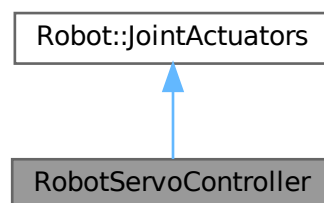
Hardware interface for servo motor control via PCA9685 PWM driver.

```
#include <ServoActuator.h>
```

Inheritance diagram for RobotServoController:



Collaboration diagram for RobotServoController:



Public Member Functions

- **RobotServoController** ([TwoWire](#) &[Wire_](#), `const std::vector< actuator\_parameters > ¶meters_`)
Initializes servo controller with I2C bus and calibration parameters.
- **void init** ()
Initializes PWM driver hardware and configures servo frequency.
- **bool writeAngles** (`const float *angles`, `std::size_t n`)
Writes joint angles to all servo actuators.

Public Member Functions inherited from [Robot::JointActuators](#)

- **JointActuators** ()
Default constructor.

7.6.1 Detailed Description

Hardware interface for servo motor control via PCA9685 PWM driver.

Implements the JointActuators interface to convert computed joint angles into PWM control signals for servo motors.

Handles:

- Joint angle to PWM conversion with calibration parameters
- Servo frequency initialization and PWM channel configuration
- Joint angle limit enforcement to prevent hardware damage

Each joint requires:

- Minimum and maximum angle constraints
- Calibration gain and offset factors

Works with STM32 and I2C-based PCA9685 16-channel PWM expander for simultaneous multi-servo control with precise timing.

7.6.2 Constructor & Destructor Documentation

7.6.2.1 RobotServoController()

```
RobotServoController::RobotServoController (
    TwoWire & Wire\_,
    const std::vector< actuator\_parameters > & parameters_ ) [inline]
```

Initializes servo controller with I2C bus and calibration parameters.

Parameters

Wire_	Reference to the I2C bus (TwoWire) for PCA9685 communication
parameters ↔	Vector of actuator_parameters (one per joint) containing:
—	• minAngle, maxAngle: joint limits in radians • gain, offset: calibration factors for angle-to-PWM conversion

Does not take ownership of the [Wire_](#) reference; caller must manage the I2C bus. Parameters are copied into the controller on construction.

7.6.3 Member Function Documentation

7.6.3.1 init()

```
void RobotServoController::init ( ) [inline]
```

Initializes PWM driver hardware and configures servo frequency.

Operations performed:

1. Reset all PCA9685 devices on the I2C bus
2. Initialize PCA9685 registers and communication
3. Set PWM frequency to standard servo frequency (typically 50 Hz)

Must be called once during system initialization before sending commands to servos. Typically invoked in the main [setup\(\)](#) or initialization task.

7.6.3.2 writeAngles()

```
bool RobotServoController::writeAngles (
    const float * angles,
    std::size_t n ) [inline], [virtual]
```

Writes joint angles to all servo actuators.

Parameters

<i>angles</i>	Array of joint angles in radians
<i>n</i>	Number of joints (array size; should match configured DOF)

Returns

true if all angles were within valid ranges and successfully written to hardware, false if any angle exceeded its joint limits (no angles are written in this case)

Conversion process for each joint *i*:

1. Check: $\text{angle_min} \leq \text{angles}[i] \leq \text{angle_max}$
2. If violated: abort and return false (fail-safe behavior)
3. If valid: convert to degrees: $\text{deg_i} = (\text{angles}[i] - \text{offset_i}) * \text{gain_i} * 180/\pi$
4. Map degrees to PWM pulse width using servo calibration
5. Write PWM value to PCA9685 channel *i*

Note

Operates in a fail-safe manner: if ANY angle violates its constraints, no PWM values are written. This prevents partial joint movements that could cause unsafe arm configurations.

Implements [Robot::JointActuators](#).

The documentation for this class was generated from the following file:

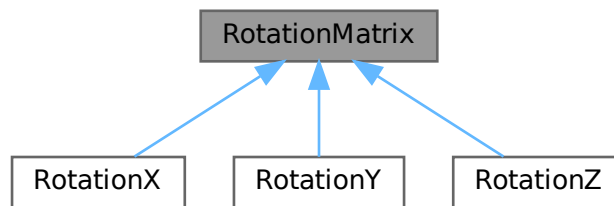
- TP26-Roboticke-rameno/[ServoActuator.h](#)

7.7 RotationMatrix Class Reference

Abstract base class for joint rotation matrix generation.

```
#include <geometry.h>
```

Inheritance diagram for RotationMatrix:



Public Member Functions

- `virtual ~RotationMatrix ()=default`
Virtual destructor.
- `virtual Matrix< 3, 3 > operator() (float theta) const =0`
Compute rotation matrix from angle.
- `virtual Matrix< 3, 3 > d (float theta) const =0`
Compute derivative of rotation matrix with respect to angle.
- `virtual void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const =0`
Compute both rotation matrix and derivative in one call.

7.7.1 Detailed Description

Abstract base class for joint rotation matrix generation.

Defines an interface for generating rotation matrices in a fixed axis (X, Y, or Z) and computing their derivatives with respect to joint angles.

Implementations are used in robot kinematics solvers to:

- Compute cumulative rotation transformations
- Calculate Jacobian matrices for inverse kinematics
- Perform forward and inverse kinematic analyses

7.7.2 Member Function Documentation

7.7.2.1 d()

```
virtual Matrix< 3, 3 > RotationMatrix::d (
    float theta ) const [pure virtual]
```

Compute derivative of rotation matrix with respect to angle.

Parameters

<i>theta</i>	Joint angle in radians
--------------	------------------------

Returns

3×3 derivative matrix: $dR/d\theta$

Implemented in [RotationX](#), [RotationY](#), and [RotationZ](#).

7.7.2.2 operator() [1/2]

```
virtual Matrix< 3, 3 > RotationMatrix::operator() (
    float theta ) const [pure virtual]
```

Compute rotation matrix from angle.

Parameters

<i>theta</i>	Joint angle in radians
--------------	------------------------

Returns

3x3 rotation matrix for this joint

Implemented in [RotationX](#), [RotationY](#), and [RotationZ](#).

7.7.2.3 operator() [2/2]

```
virtual void RotationMatrix::operator() (
    float theta,
    Matrix< 3, 3 > R,
    Matrix< 3, 3 > & dR ) const [pure virtual]
```

Compute both rotation matrix and derivative in one call.

Parameters

<i>theta</i>	Joint angle in radians
<i>R</i>	[out] Computed rotation matrix (passed by value, updated in call)
<i>dR</i>	[out] Computed derivative matrix (passed by reference)

Note

The parameter R is intentionally passed by value for API consistency. Only dR is modified within this function. This is an optimization pattern where both computations share sin/cos evaluation.

Implemented in [RotationX](#), [RotationY](#), and [RotationZ](#).

The documentation for this class was generated from the following file:

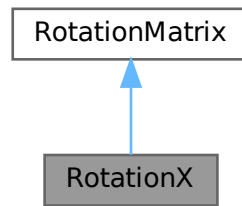
- TP26-Roboticke-rameno/[geometry.h](#)

7.8 RotationX Class Reference

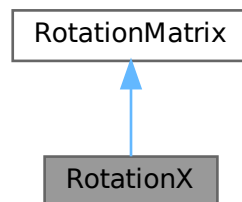
Rotation in X-axis (right-hand rule: thumb along +X)

```
#include <geometry.h>
```

Inheritance diagram for RotationX:



Collaboration diagram for RotationX:



Public Member Functions

- `Matrix< 3, 3 > operator() (float theta) const override`
Compute X-axis rotation matrix.
- `Matrix< 3, 3 > d (float theta) const override`
Compute derivative of X-axis rotation matrix.
- `void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const override`
Compute both X-axis rotation matrix and derivative.

Public Member Functions inherited from `RotationMatrix`

- `virtual ~RotationMatrix ()=default`
Virtual destructor.

7.8.1 Detailed Description

Rotation in X-axis (right-hand rule: thumb along +X)

Concrete implementation of `RotationMatrix` for rotations in the X-axis. Common for shoulder/base joints in robotic manipulators.

7.8.2 Member Function Documentation

7.8.2.1 d()

```
Matrix< 3, 3 > RotationX::d (
    float theta ) const [inline], [override], [virtual]
```

Compute derivative of X-axis rotation matrix.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 derivative matrix: $dR_x/d\theta$

Implements [RotationMatrix](#).

7.8.2.2 operator>() [1/2]

```
Matrix< 3, 3 > RotationX::operator() (
    float theta ) const [inline], [override], [virtual]
```

Compute X-axis rotation matrix.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 rotation matrix in X-axis

Implements [RotationMatrix](#).

7.8.2.3 operator>() [2/2]

```
void RotationX::operator() (
    float theta,
    Matrix< 3, 3 > R,
    Matrix< 3, 3 > & dR ) const [inline], [override], [virtual]
```

Compute both X-axis rotation matrix and derivative.

Parameters

<i>theta</i>	Rotation angle in radians
<i>R</i>	[out] Computed rotation matrix
<i>dR</i>	[out] Computed derivative matrix

Implements [RotationMatrix](#).

The documentation for this class was generated from the following file:

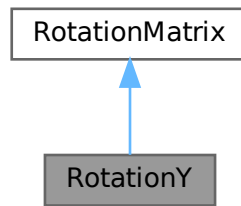
- TP26-Roboticke-rameno/[geometry.h](#)

7.9 RotationY Class Reference

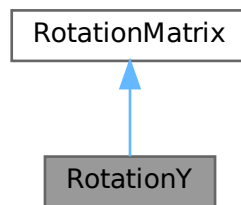
Rotation in Y-axis (right-hand rule: thumb along +Y)

```
#include <geometry.h>
```

Inheritance diagram for RotationY:



Collaboration diagram for RotationY:



Public Member Functions

- `Matrix< 3, 3 > operator() (float theta) const override`
Compute Y-axis rotation matrix.
- `Matrix< 3, 3 > d (float theta) const override`
Compute derivative of Y-axis rotation matrix.
- `void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const override`
Compute both Y-axis rotation matrix and derivative.

Public Member Functions inherited from `RotationMatrix`

- `virtual ~RotationMatrix ()=default`
Virtual destructor.

7.9.1 Detailed Description

Rotation in Y-axis (right-hand rule: thumb along +Y)

Concrete implementation of `RotationMatrix` for rotations in the Y-axis. Common for elbow and wrist pitch joints in robotic manipulators.

7.9.2 Member Function Documentation

7.9.2.1 d()

```
Matrix< 3, 3 > RotationY::d (
    float theta ) const [inline], [override], [virtual]
```

Compute derivative of Y-axis rotation matrix.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 derivative matrix: $dR_y/d\theta$

Implements [RotationMatrix](#).

7.9.2.2 operator() [1/2]

```
Matrix< 3, 3 > RotationY::operator() (
    float theta ) const [inline], [override], [virtual]
```

Compute Y-axis rotation matrix.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 rotation matrix in Y-axis

Implements [RotationMatrix](#).

7.9.2.3 operator() [2/2]

```
void RotationY::operator() (
    float theta,
    Matrix< 3, 3 > R,
    Matrix< 3, 3 > & dR ) const [inline], [override], [virtual]
```

Compute both Y-axis rotation matrix and derivative.

Parameters

<i>theta</i>	Rotation angle in radians
<i>R</i>	[out] Computed rotation matrix
<i>dR</i>	[out] Computed derivative matrix

Implements [RotationMatrix](#).

The documentation for this class was generated from the following file:

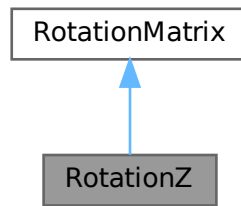
- TP26-Roboticke-rameno/[geometry.h](#)

7.10 RotationZ Class Reference

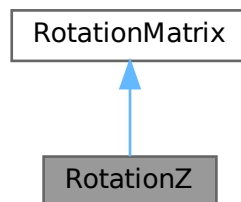
Rotation in Z-axis (right-hand rule: thumb along +Z)

```
#include <geometry.h>
```

Inheritance diagram for RotationZ:



Collaboration diagram for RotationZ:



Public Member Functions

- `Matrix< 3, 3 > operator() (float theta) const override`
Compute Z-axis rotation matrix.
- `Matrix< 3, 3 > d (float theta) const override`
Compute derivative of Z-axis rotation matrix.
- `void operator() (float theta, Matrix< 3, 3 > R, Matrix< 3, 3 > &dR) const override`
Compute both Z-axis rotation matrix and derivative.

Public Member Functions inherited from `RotationMatrix`

- `virtual ~RotationMatrix ()=default`
Virtual destructor.

7.10.1 Detailed Description

Rotation in Z-axis (right-hand rule: thumb along +Z)

Concrete implementation of `RotationMatrix` for rotations in the Z-axis. Common for waist/azimuth joints and wrist yaw in robotic manipulators.

7.10.2 Member Function Documentation

7.10.2.1 d()

```
Matrix< 3, 3 > RotationZ::d (
    float theta ) const [inline], [override], [virtual]
```

Compute derivative of Z-axis rotation matrix.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 derivative matrix: $dR_z/d\theta$

Implements [RotationMatrix](#).

7.10.2.2 operator() [1/2]

```
Matrix< 3, 3 > RotationZ::operator() (
    float theta ) const [inline], [override], [virtual]
```

Compute Z-axis rotation matrix.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 rotation matrix in Z-axis

Implements [RotationMatrix](#).

7.10.2.3 operator() [2/2]

```
void RotationZ::operator() (
    float theta,
    Matrix< 3, 3 > R,
    Matrix< 3, 3 > & dR ) const [inline], [override], [virtual]
```

Compute both Z-axis rotation matrix and derivative.

Parameters

<i>theta</i>	Rotation angle in radians
<i>R</i>	[out] Computed rotation matrix
<i>dR</i>	[out] Computed derivative matrix

Implements [RotationMatrix](#).

The documentation for this class was generated from the following file:

- TP26-Roboticke-rameno/[geometry.h](#)

Chapter 8

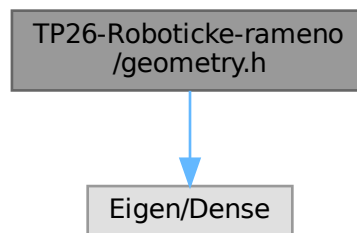
File Documentation

8.1 TP26-Roboticke-rameno/geometry.h File Reference

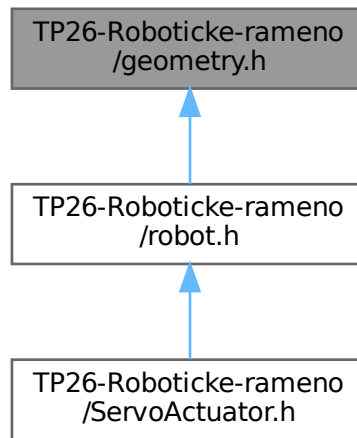
Rotation matrices and derivatives for robot kinematics.

```
#include "Eigen/Dense"
```

Include dependency graph for geometry.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [RotationMatrix](#)
Abstract base class for joint rotation matrix generation.
- class [RotationX](#)
Rotation in X-axis (right-hand rule: thumb along +X)
- class [RotationY](#)
Rotation in Y-axis (right-hand rule: thumb along +Y)
- class [RotationZ](#)
Rotation in Z-axis (right-hand rule: thumb along +Z)

Typedefs

- `template<int Rows, int Cols>`
`using Matrix = Eigen::Matrix< float, Rows, Cols >`
Type alias for fixed-size Eigen matrix of floats.

Functions

Rotation matrix generation and derivatives

Each set of functions provides:

- *Rotation matrix from sine and cosine values.*
- *Rotation matrix from an angle (radians).*
- *Derivative of rotation matrix w.r.t. angle.*

Rotations follow the right-hand rule.

- `Matrix< 3, 3 > rotX (float s\_theta, float c\_theta)`
Rotation matrix in the X-axis from pre-computed sine and cosine.
- `Matrix< 3, 3 > rotX (float theta)`
Rotation matrix in the X-axis from angle in radians.
- `Matrix< 3, 3 > drotX (float s\_theta, float c\_theta)`

- Derivative of X-axis rotation matrix with respect to angle from pre-computed sin/cos.*
- `Matrix< 3, 3 > drotX (float theta)`
- Derivative of X-axis rotation matrix with respect to angle in radians.*
- `Matrix< 3, 3 > rotY (float s_theta, float c_theta)`
- Rotation matrix in the Y-axis from pre-computed sine and cosine.*
- `Matrix< 3, 3 > rotY (float theta)`
- Rotation matrix in the Y-axis from angle in radians.*
- `Matrix< 3, 3 > drotY (float s_theta, float c_theta)`
- Derivative of Y-axis rotation matrix with respect to angle from pre-computed sin/cos.*
- `Matrix< 3, 3 > drotY (float theta)`
- Derivative of Y-axis rotation matrix with respect to angle in radians.*
- `Matrix< 3, 3 > rotZ (float s_theta, float c_theta)`
- Rotation matrix in the Z-axis from pre-computed sine and cosine.*
- `Matrix< 3, 3 > rotZ (float theta)`
- Rotation matrix in the Z-axis from angle in radians.*
- `Matrix< 3, 3 > drotZ (float s_theta, float c_theta)`
- Derivative of Z-axis rotation matrix with respect to angle from pre-computed sin/cos.*
- `Matrix< 3, 3 > drotZ (float theta)`
- Derivative of Z-axis rotation matrix with respect to angle in radians.*

8.1.1 Detailed Description

Rotation matrices and derivatives for robot kinematics.

8.1.2 Function Documentation

8.1.2.1 drotX() [1/2]

```
Matrix< 3, 3 > drotX (
    float s_theta,
    float c_theta )
```

Derivative of X-axis rotation matrix with respect to angle from pre-computed sin/cos.

Parameters

<code>s_theta</code>	Sine of the rotation angle
<code>c_theta</code>	Cosine of the rotation angle

Returns

3×3 derivative matrix: $dR_x/d\theta$

Represents the rate of change of the rotation matrix as the angle changes. Used in Jacobian computations for inverse kinematics.

8.1.2.2 drotX() [2/2]

```
Matrix< 3, 3 > drotX (
    float theta )
```

Derivative of X-axis rotation matrix with respect to angle in radians.

Parameters

<code>theta</code>	Rotation angle in radians
--------------------	---------------------------

Returns

3×3 derivative matrix: $dR_x/d\theta$

8.1.2.3 drotY() [1/2]

```
Matrix< 3, 3 > drotY (
    float s_theta,
    float c_theta )
```

Derivative of Y-axis rotation matrix with respect to angle from pre-computed sin/cos.

Parameters

<i>s_theta</i>	Sine of the rotation angle
<i>c_theta</i>	Cosine of the rotation angle

Returns

3×3 derivative matrix: $dR_y/d\theta$

Represents the rate of change of the rotation matrix as the angle changes. Used in Jacobian computations for inverse kinematics.

8.1.2.4 drotY() [2/2]

```
Matrix< 3, 3 > drotY (
    float theta )
```

Derivative of Y-axis rotation matrix with respect to angle in radians.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 derivative matrix: $dR_y/d\theta$

8.1.2.5 drotZ() [1/2]

```
Matrix< 3, 3 > drotZ (
    float s_theta,
    float c_theta )
```

Derivative of Z-axis rotation matrix with respect to angle from pre-computed sin/cos.

Parameters

<i>s_theta</i>	Sine of the rotation angle
<i>c_theta</i>	Cosine of the rotation angle

Returns

3×3 derivative matrix: $dR_z/d\theta$

Represents the rate of change of the rotation matrix as the angle changes. Used in Jacobian computations for inverse kinematics.

8.1.2.6 drotZ() [2/2]

```
Matrix< 3, 3 > drotZ (
    float theta )
```

Derivative of Z-axis rotation matrix with respect to angle in radians.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 derivative matrix: $dR_z/d\theta$

8.1.2.7 rotX() [1/2]

```
Matrix< 3, 3 > rotX (
    float s_theta,
    float c_theta )
```

Rotation matrix in the X-axis from pre-computed sine and cosine.

Parameters

<i>s_theta</i>	Sine of the rotation angle
<i>c_theta</i>	Cosine of the rotation angle

Returns

3×3 rotation matrix for rotation in X-axis

Avoids redundant trigonometric computations when both sin and cos are available.

8.1.2.8 rotX() [2/2]

```
Matrix< 3, 3 > rotX (
    float theta )
```

Rotation matrix in the X-axis from angle in radians.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 rotation matrix for rotation in X-axis (right-hand rule)

8.1.2.9 rotY() [1/2]

```
Matrix< 3, 3 > rotY (
    float s_theta,
    float c_theta )
```

Rotation matrix in the Y-axis from pre-computed sine and cosine.

Parameters

<i>s_theta</i>	Sine of the rotation angle
<i>c_theta</i>	Cosine of the rotation angle

Returns

3×3 rotation matrix for rotation in Y-axis

Avoids redundant trigonometric computations when both sin and cos are available.

8.1.2.10 rotY() [2/2]

```
Matrix< 3, 3 > rotY (
    float theta )
```

Rotation matrix in the Y-axis from angle in radians.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 rotation matrix for rotation in Y-axis (right-hand rule)

8.1.2.11 rotZ() [1/2]

```
Matrix< 3, 3 > rotZ (
    float s_theta,
    float c_theta )
```

Rotation matrix in the Z-axis from pre-computed sine and cosine.

Parameters

<i>s_theta</i>	Sine of the rotation angle
<i>c_theta</i>	Cosine of the rotation angle

Returns

3×3 rotation matrix for rotation in Z-axis

Avoids redundant trigonometric computations when both sin and cos are available.

8.1.2.12 rotZ() [2/2]

```
Matrix< 3, 3 > rotZ (
    float theta )
```

Rotation matrix in the Z-axis from angle in radians.

Parameters

<i>theta</i>	Rotation angle in radians
--------------	---------------------------

Returns

3×3 rotation matrix for rotation in Z-axis (right-hand rule)

8.2 geometry.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006
00007 #include "Eigen/Dense"
00008
00009
00013 template<int Rows, int Cols>
00014 using Matrix = Eigen::Matrix<float, Rows, Cols>;
00015
00027
00036 Matrix<3,3> rotX(float s_theta, float c_theta)
00037 {
00038     Matrix<3, 3> R;
00039     R << 1.0, 0, 0,
00040         0, c_theta, -s_theta,
00041         0, s_theta, c_theta;
00042     return R;
00043 }
00044
00050 Matrix<3,3> rotX(float theta)
00051 {
00052     return rotX(sin(theta), cos(theta));
00053 }
00054
00064 Matrix<3,3> drotX(float s_theta, float c_theta)
00065 {
00066     Matrix<3, 3> R;
00067     R << 0, 0, 0,
00068         0, -s_theta, -c_theta,
00069         0, c_theta, -s_theta;
00070     return R;
00071 }
00072
00078 Matrix<3,3> drotX(float theta)
00079 {
00080     return drotX(sin(theta), cos(theta));
00081 }
00082
00091 Matrix<3,3> rotY(float s_theta, float c_theta)
00092 {
00093     Matrix<3, 3> R;
00094     R << c_theta, 0, s_theta,
00095         0, 1.0, 0,
00096         -s_theta, 0, c_theta;
00097     return R;
00098 }
00099
00105 Matrix<3,3> rotY(float theta)
00106 {
00107     return rotY(sin(theta), cos(theta));
00108 }
00109
00119 Matrix<3,3> drotY(float s_theta, float c_theta)
00120 {
00121     Matrix<3, 3> R;
00122     R << -s_theta, 0, c_theta,
00123         0, 0, 0,
00124         -c_theta, 0, -s_theta;
00125     return R;
00126 }
00127
00133 Matrix<3,3> drotY(float theta)
00134 {
00135     return drotY(sin(theta), cos(theta));
00136 }
00137
00146 Matrix<3,3> rotZ(float s_theta, float c_theta)
00147 {
00148     Matrix<3, 3> R;
00149     R << c_theta, -s_theta, 0,
00150         s_theta, c_theta, 0,
00151         0, 0, 1.0;
00152     return R;
00153 }
00154
00160 Matrix<3,3> rotZ(float theta)
00161 {

```

```

00162     return rotZ(sin(theta), cos(theta));
00163 }
00164
00174 Matrix<3,3> drotZ(float s_theta, float c_theta)
00175 {
00176     Matrix<3, 3> R;
00177     R << -s_theta, -c_theta, 0,
00178         c_theta, -s_theta, 0,
00179         0, 0, 0;
00180     return R;
00181 }
00182
00188 Matrix<3,3> drotZ(float theta)
00189 {
00190     return drotZ(sin(theta), cos(theta));
00191 }
00192
00193
00206 class RotationMatrix {
00207 public:
00211     virtual ~RotationMatrix() = default;
00212
00218     virtual Matrix<3,3> operator()(float theta) const = 0;
00219
00225     virtual Matrix<3,3> d(float theta) const = 0;
00226
00237     virtual void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const = 0;
00238 };
00239
00247 class RotationX : public RotationMatrix {
00248 public:
00254     Matrix<3,3> operator()(float theta) const override {
00255         float s = sin(theta), c = cos(theta);
00256         return rotX(s, c);
00257     }
00258
00264     Matrix<3,3> d(float theta) const override {
00265         float s = sin(theta), c = cos(theta);
00266         return drotX(s, c);
00267     }
00268
00275     void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const override {
00276         float s = sin(theta), c = cos(theta);
00277         R = rotX(s, c);
00278         dR = drotX(s, c);
00279     }
00280 };
00281
00289 class RotationY : public RotationMatrix {
00290 public:
00296     Matrix<3,3> operator()(float theta) const override {
00297         float s = sin(theta), c = cos(theta);
00298         return rotY(s, c);
00299     }
00300
00306     Matrix<3,3> d(float theta) const override {
00307         float s = sin(theta), c = cos(theta);
00308         return drotY(s, c);
00309     }
00310
00317     void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const override {
00318         float s = sin(theta), c = cos(theta);
00319         R = rotY(s, c);
00320         dR = drotY(s, c);
00321     }
00322 };
00323
00331 class RotationZ : public RotationMatrix {
00332 public:
00338     Matrix<3,3> operator()(float theta) const override {
00339         float s = sin(theta), c = cos(theta);
00340         return rotZ(s, c);
00341     }
00342
00348     Matrix<3,3> d(float theta) const override {
00349         float s = sin(theta), c = cos(theta);
00350         return drotZ(s, c);
00351     }
00352
00359     void operator()(float theta, Matrix<3,3> R, Matrix<3,3> &dR) const override {
00360         float s = sin(theta), c = cos(theta);
00361         R = rotZ(s, c);
00362         dR = drotZ(s, c);
00363     }
00364 };
00365
00366

```


00367
00368

8.3 TP26-Roboticke-rameno/robot.h File Reference

[Robot](#) kinematics and actuator interfaces.

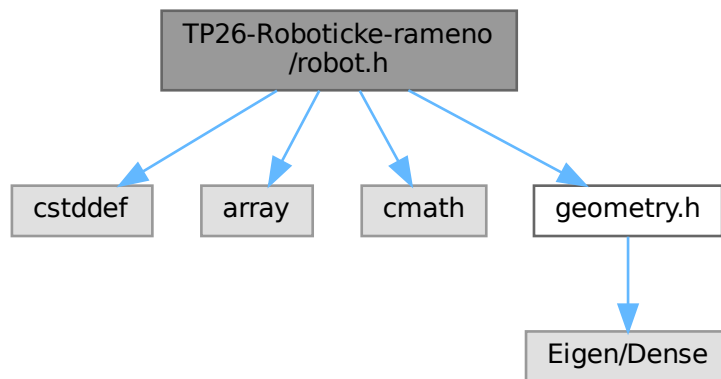
```
#include <cstdint>
```

```
#include <array>
```

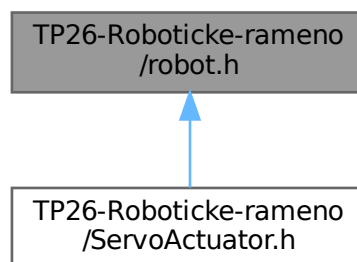
```
#include <cmath>
```

```
#include "geometry.h"
```

Include dependency graph for robot.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [actuator_parameters](#)
Joint actuator configuration and calibration parameters.
- class [Robot::RobotKinematicsBase< DOF >](#)
Abstract base class for robot kinematics solvers.
- class [Robot::JointActuators](#)

Abstract base class for joint actuation hardware.

- class `Robot::RobotArm< DOF >`

High-level robot arm control interface.

- class `Robot::RobotKinematics< DOF >`

Concrete kinematics solver using Jacobian-based inverse kinematics.

Namespaces

- namespace `Robot`

Core robotics control and kinematics namespace.

Macros

- `#define M_PI 3.14159265358979323846f`

Typedefs

- `template<int Rows, int Cols>`
`using Matrix = Eigen::Matrix< float, Rows, Cols >`

8.3.1 Detailed Description

`Robot` kinematics and actuator interfaces.

8.4 robot.h

[Go to the documentation of this file.](#)

```
00001
00005 #pragma once
00006 #include <cstdint>
00007 #include <array>
00008 #include <cmath>
00009 #pragma once
00010
00011 #ifndef M_PI
00012 #define M_PI 3.14159265358979323846f
00013 #endif
00014
00015 #include "geometry.h"
00016
00017 template<int Rows, int Cols>
00018 using Matrix = Eigen::Matrix<float, Rows, Cols>;
00019
00027 typedef struct
00028 {
00029     float minAngle;
00030     float maxAngle;
00031     float gain;
00032     float offset;
00033 } actuator_parameters;
00034
00035
00043 namespace Robot {
00044
00052     static inline Matrix<3,1> T(float l) {
00053         Matrix<3,1> t;
00054         t << 0.0f, 0.0f, l;
00055         return t;
00056     }
00057
00067 template<std::size_t DOF>
00068 class RobotKinematicsBase {
00069 public:
00070
00083     virtual bool SolveIK(
00084         const Matrix<3, 1>& pr,
00085         const Matrix<3, 3>& Rr,
00086         Matrix<DOF, 1>& theta_,
00087         float tol_pos,
00088         float tol_ori,
00089         float lambda,
00090         int max_iter) = 0;
```

```

00091
00097     virtual std::pair<Matrix<3, 1>, Matrix<3, 3>> forwardKinematics(const Matrix<DOF, 1>& theta) const =
00098         0;
00099 };
00100
00101
00110 class JointActuators {
00111 public:
00115     JointActuators() {}
00116
00124     virtual bool writeAngles(const float*, std::size_t) = 0;
00125 };
00126
00137 template<std::size_t DOF>
00138 class RobotArm {
00139 public:
00140
00148     RobotArm(RobotKinematicsBase<DOF>* robot_kinematics, JointActuators* actuators)
00149         : robot_kinematics_(robot_kinematics), jointAngles_(Matrix<DOF, 1>::Zero()), actuators_(actuators)
00150     {
00151
00155     ~RobotArm() = default;
00156
00166     bool setJointAngles(const Matrix<DOF, 1>& angles) {
00167
00168         if (!actuators_>writeAngles(angles.data(), DOF)) return false;
00169         jointAngles_ = angles;
00170         return true;
00171     }
00172
00178     float getJointAngle(std::size_t index) const {
00179         return jointAngles_(index);
00180     }
00181
00186     Matrix<DOF, 1> getJointAngles() const {
00187         return jointAngles_;
00188     }
00189
00195     std::pair<Matrix<3, 1>, Matrix<3, 3>> getCurrentEffectorPositionOrientation() const {
00196         return robot_kinematics_>forwardKinematics(jointAngles_);
00197     }
00198
00210     bool goTo(const Matrix<3, 1>& pr, const Matrix<3, 3>& Rr) {
00211         Matrix<DOF, 1> theta = jointAngles_;
00212         bool res = robot_kinematics_>SolveIK(pr, Rr, theta, 1e-5f, 1e-3f, 1e-2, 50);
00213
00214         if (res) {
00215             res = setJointAngles(theta);
00216         }
00217         return res;
00218     }
00219
00220
00221
00222 protected:
00223     JointActuators* actuators_;
00224     Matrix<DOF, 1> jointAngles_;
00225     RobotKinematicsBase<DOF>* robot_kinematics_;
00226 };
00227
00238 template<std::size_t DOF>
00239 class RobotKinematics : public RobotKinematicsBase<DOF>
00240 {
00241 public:
00242
00249     RobotKinematics(const std::array<Matrix<3, 1>, DOF>& L_,
00250                     const std::array<const RotationMatrix*, DOF>& R_)
00251         : L(L_), R_func(R_) {}
00252
00260     std::pair<Matrix<3, 1>, Matrix<3, 3>>
00261     forwardKinematics(const Matrix<DOF, 1>& theta) const override
00262     {
00263         // Iterujeme od posledného kĺbu dovnútra smerom k základni
00264         Matrix<3, 1> vekt = Matrix<3, 1>::Zero();
00265         for (int i = (int)DOF - 1; i >= 0; --i)
00266         {
00267             vekt = (*R_func[i])(theta(i)) * (L[i] + vekt);
00268         }
00269
00270         // Orientácia efektora: R1 * R2 * ... * Rn
00271         Matrix<3, 3> R_ee = Matrix<3, 3>::Identity();
00272         for (std::size_t i = 0; i < DOF; ++i)
00273             R_ee = R_ee * (*R_func[i])(theta(i));
00274

```

```

00275         return { vekt, R_ee };
00276     }
00277
00287 std::array<Matrix<3, 3>, DOF> getRotDerivatives(const Matrix<DOF, 1>& theta) const {
00288     std::array<Matrix<3, 3>, DOF> mat_all; // Pole pre výsledné parciálne derivácie
00289     int n = DOF;
00290     Matrix<3, 3> f_mat = Matrix<3, 3>::Identity();
00291     Matrix<3, 3> b_mat = Matrix<3, 3>::Identity();
00292
00293     // Prvý cyklus: Výpočet celkového dopredného súčinu matic
00294     // Týmto f_mat naakumuluje súčin R_0 * R_1 * ... * R_{n-2}
00295     for (int i = n - 2; i >= 0; i--) {
00296         f_mat = (*R_func[i])(theta(i)) * f_mat;
00297     }
00298
00299     // Druhý cyklus: Výpočet derivácií smerom od konca k začiatku
00300     for (int j = n - 1; j >= 0; j--) {
00301         if (j < n - 1) {
00302             b_mat = (*R_func[j + 1])(theta(j + 1)) * b_mat;
00303
00304             // "Odstraňovanie" aktuálnej matice zľava pomocou transpozície ( $R^{-1} = R^T$ )
00305             f_mat = f_mat * (*R_func[j])(theta(j)).transpose();
00306         }
00307
00308         // Samotný výpočet derivácie pre j-ty kĺb
00309         // Pre prístup k derivácii voláme metódu d() cez pointer z tvojho pol'a R_func
00310         mat_all[j] = f_mat * R_func[j]->d(theta(j)) * b_mat;
00311     }
00312
00313     return mat_all;
00314 }
00315
00327 Matrix<3, DOF> getJacobian(const Matrix<DOF, 1>& theta) const {
00328     int n = DOF; // Počet kĺbov
00329     Matrix<3, DOF> jakob_mat = Matrix<3, DOF>::Zero(); // Výsledná Jakobián matica (3 riadky, n
00330     stĺpcov)
00331     Matrix<3, 3> matica = Matrix<3, 3>::Identity(); // [1 0 0; 0 1 0; 0 0 1]
00332     Matrix<3, 1> pom_vekt = L[n - 1]; // Inicializácia T[n] v pseudokóde
00333
00334     // Prvý cyklus: od i = n-1 do 1 (1-based) -> od i = n-2 po 0 (0-based)
00335     // Kumuluje doprednú kinematiku (súčin rotácií od n-2 po 0)
00336     for (int i = n - 2; i >= 0; i--) {
00337         matica = (*R_func[i])(theta(i)) * matica;
00338     }
00339
00340     // Druhý cyklus: od i = n do 1 (1-based) -> od i = n-1 po 0 (0-based)
00341     for (int i = n - 1; i >= 0; i--) {
00342
00343         Matrix<3, 1> vekt = L[n - 1]; // T[n] z pseudokódu
00344
00345         if (i < n - 1) { // Podmienka (n > i) z pseudokódu v 0-based svete
00346
00347             // "Odstraňovanie" rotácie pre aktuálny kĺb
00348             matica = matica * (*R_func[i])(theta(i)).transpose();
00349
00350             // Akumulácia translačných vektorov od konca (efektora) ku kĺbu 'i'
00351             pom_vekt = L[i] + (*R_func[i + 1])(theta(i + 1)) * pom_vekt;
00352
00353             vekt = pom_vekt;
00354         }
00355
00356         // Výpočet samotného stĺpca Jakobiánu
00357         // matica: ( $R_1 * R_2 \dots * R_{i-1}$ )
00358         // dR_i: parciálna derivácia i-teho kĺbu
00359         // vekt: ( $T_i + R_{i+1} * (T_{i+1} + \dots)$ )
00360         vekt = matica * R_func[i]->d(theta(i)) * vekt;
00361
00362         // Uloženie výsledného vektora do i-teho stĺpca matice Jakobiánu
00363         jakob_mat.col(i) = vekt;
00364     }
00365
00366     return jakob_mat;
00367 }
00368
00390 bool SolveIK(
00391     const Matrix<3, 1>& pr,
00392     const Matrix<3, 3>& Rr,
00393     Matrix<DOF, 1>& theta,
00394     float tol_pos,
00395     float tol_ori,
00396     float lambda,
00397     int max_iter) override
00398 {
00399     for (int i = 0; i < max_iter; i++)
00400     {
00401         // 1. Dopredná kinematika (aktuálny stav)

```

```

00402         auto fk_res = forwardKinematics(theta);
00403         Matrix<3, 1> p0 = fk_res.first;
00404         Matrix<3, 3> R0 = fk_res.second;
00405
00406         // 2. Výpočet reziduí (chyby)
00407         Matrix<3, 1> rp = p0 - pr;
00408
00409         // Výpočet matice chyby orientácie E = R0 * Rr^T
00410         Matrix<3, 3> E = R0 * Rr.transpose();
00411
00412         // Extrakcia chyby orientácie podľa tvojho pseudokódu
00413         Matrix<3, 1> r0;
00414         r0(0) = E(1, 0);
00415         r0(1) = E(2, 0);
00416         r0(2) = E(2, 1);
00417
00418         // 3. Kontrola ukončenia (ak sme v tolerancii)
00419         if (rp.norm() < tol_pos && r0.norm() < tol_ori) {
00420             return true;
00421         }
00422
00423         // 4. Získanie Jakobiánov (tieto funkcie už v robot.h máš)
00424         Matrix<3, DOF> Jp = getJacobian(theta);
00425         std::array<Matrix<3, 3>, DOF> dEr_R = getRotDerivatives(theta);
00426
00427         // Zostavenie plného Jakobiánu 6xDOF
00428         Matrix<6, DOF> J = Matrix<6, DOF>::Zero();
00429
00430         // Horná časť (poloha)
00431         J.template block<3, DOF>(0, 0) = Jp;
00432
00433         // Dolná časť (orientácia - podľa dE_j = dR_j * Rr^T)
00434         for (std::size_t j = 0; j < DOF; j++) {
00435             Matrix<3, 3> dE_j = dEr_R[j] * Rr.transpose();
00436
00437             J(3, j) = dE_j(1, 0);
00438             J(4, j) = dE_j(2, 0);
00439             J(5, j) = dE_j(2, 1);
00440         }
00441
00442         // 5. Regularizácia (Levenberg-Marquardt štýl)
00443         for (std::size_t k = 0; k < DOF; k++) {
00444             J(k, k) += lambda;
00445         }
00446
00447         // 6. Zostavenie vektora záporných reziduí
00448         Matrix<6, 1> minus_r;
00449         minus_r.template block<3, 1>(0, 0) = -rp;
00450         minus_r.template block<3, 1>(3, 0) = -r0;
00451
00452         // 7. Výpočet zmeny uhlov (Riešenie sústavy J * delta_theta = minus_r)
00453         // Pre 6x6 maticu je ColPivHouseholderQR veľmi stabilná voľba
00454         Matrix<DOF, 1> delta_theta = J.colPivHouseholderQR().solve(minus_r);
00455
00456         // 8. Update
00457         theta += delta_theta;
00458
00459         // --- PRIDANÉ: Normalizácia uhlov do intervalu [-pi, pi] na záver iterácie ---
00460         for (std::size_t j = 0; j < DOF; ++j) {
00461             // std::remainder vráti zvyšok po delení, centrovany okolo 0
00462             theta(j) = std::remainder(theta(j), 2.0f * M_PI);
00463         }
00464         // -----
00465     }
00466
00467     return false; // Nepodarilo sa nájsť riešenie v rámci max_iter
00468 }
00469
00470
00471 private:
00472     std::array<Matrix<3, 1>, DOF> L;
00473     std::array<const RotationMatrix*, DOF> R_func;
00474 };
00475
00476 } // namespace Robot

```

8.5 TP26-Roboticke-rameno/ServoActuator.h File Reference

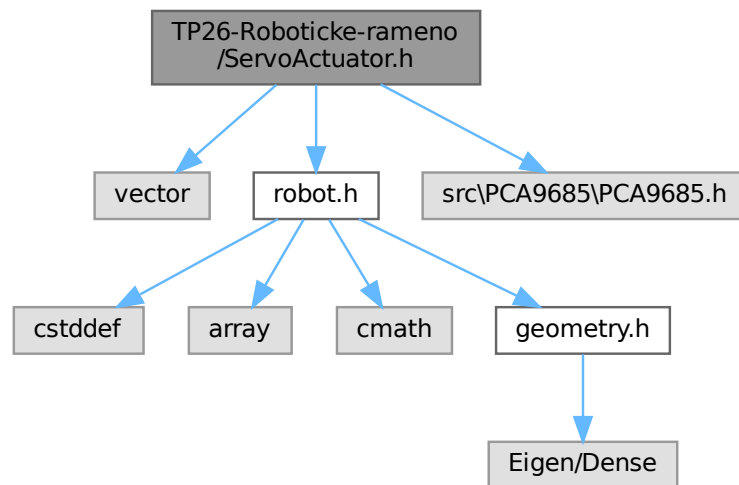
Servo actuator controller using PCA9685.

```

#include <vector>
#include "robot.h"
#include "src\PCA9685\PCA9685.h"

```

Include dependency graph for ServoActuator.h:



Classes

- class [RobotServoController](#)

Hardware interface for servo motor control via PCA9685 PWM driver.

8.5.1 Detailed Description

Servo actuator controller using PCA9685.

8.6 ServoActuator.h

[Go to the documentation of this file.](#)

```

00001
00005 #pragma once
00006
00007 #include <vector>
00008
00009 #include "robot.h"
00010 #include "src\\PCA9685\\PCA9685.h"
00011
00029 class RobotServoController : public Robot::JointActuators {
00030 public:
00031
00042 RobotServoController(TwoWire& Wire_, const std::vector<actuator_parameters>& parameters_)
00043 : Robot::JointActuators(), pwmController(Wire_), parameters(parameters_) {
00044 }
00045
00057 void init() {
00058     pwmController.resetDevices();
00059     pwmController.init();
00060     pwmController.setPWMPFreqServo();
00061 }
00062
00081 bool writeAngles(const float* angles, std::size_t n) {
00082     for (int i = 0; i < n; i++) {
00083         if (angles[i] > parameters[i].maxAngle || angles[i] < parameters[i].minAngle) return false;
00084         float degrees = (angles[i] - parameters[i].offset) * parameters[i].gain * 180.0f / PI;
00085         pwmController.setChannelPWM(i, pwmServo.pwmForAngle(degrees));
00086     }
00087     return true;
00088 }
00089
00090

```

```

00091 private:
00092     PCA9685 pwmController;
00093     PCA9685_ServoEval pwmServo;
00094     std::vector<actuator_parameters> parameters;
00095 };
00096

```

8.7 TP26-Roboticke-rameno/STM32FreeRTOSConfig.h File Reference

FreeRTOS configuration for STM32 targets.

Macros

- `#define configMEMMANG_HEAP_NB` -1
- `#define configUSE_MALLOC_FAILED_HOOK` BLINK 0
- `#define configCHECK_FOR_STACK_OVERFLOW` BLINK 0
- `#define configMAX_PRIORITIES` (7)
- `#define configMINIMAL_STACK_SIZE` ((uint16_t)((uint32_t)&_Min_Stack_Size/8))
- `#define configTOTAL_HEAP_SIZE` ((size_t)((uint32_t)&_estack - (uint32_t)&_Min_Stack_Size - (uint32_t)&_end))
- `#define configISR_STACK_SIZE_WORDS` ((uint32_t)&_Min_Stack_Size/4)
- `#define configUSE_PREEMPTION` 1
- `#define configUSE_IDLE_HOOK` 1
- `#define configUSE_TICK_HOOK` 1
- `#define configCPU_CLOCK_HZ` (SystemCoreClock)
- `#define configTICK_RATE_HZ` ((TickType_t)1000)
- `#define configMAX_TASK_NAME_LEN` (16)
- `#define configUSE_TRACE_FACILITY` 0
- `#define configUSE_16_BIT_TICKS` 0
- `#define configIDLE_SHOULD_YIELD` 1
- `#define configUSE_MUTEXES` 1
- `#define configQUEUE_REGISTRY_SIZE` 8
- `#define configCHECK_FOR_STACK_OVERFLOW` 0
- `#define configUSE_RECURSIVE_MUTEXES` 0
- `#define configUSE_MALLOC_FAILED_HOOK` 0
- `#define configUSE_APPLICATION_TASK_TAG` 0
- `#define configUSE_COUNTING_SEMAPHORES` 0
- `#define configGENERATE_RUN_TIME_STATS` 0
- `#define configUSE_NEWLIB_REENTRANT` 1
- `#define configUSE_TIME_SLICING` 1
- `#define configENABLE_MPU` 0
- `#define configENABLE_FPU` 1
- `#define configENABLE_TRUSTZONE` 0
- `#define configUSE_CO_ROUTINES` 0
- `#define configMAX_CO_ROUTINE_PRIORITIES` (2)
- `#define configUSE_TIMERS` 1
- `#define configTIMER_TASK_PRIORITY` (5)
- `#define configTIMER_QUEUE_LENGTH` 10
- `#define configTIMER_TASK_STACK_DEPTH` (configMINIMAL_STACK_SIZE)
- `#define INCLUDE_vTaskPrioritySet` 1
- `#define INCLUDE_uxTaskPriorityGet` 1
- `#define INCLUDE_vTaskDelete` 1
- `#define INCLUDE_vTaskCleanUpResources` 1
- `#define INCLUDE_vTaskSuspend` 1
- `#define INCLUDE_vTaskDelayUntil` 1
- `#define INCLUDE_vTaskDelay` 1

- `#define INCLUDE_xTaskGetSchedulerState 1`
- `#define INCLUDE_uxTaskGetStackHighWaterMark 1`
- `#define INCLUDE_xTaskGetIdleTaskHandle 1`
- `#define configPRIO_BITS 4 /* 15 priority levels */`
- `#define configLIBRARY_LOWEST_INTERRUPT_PRIORITY 0xf`
- `#define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5`
- `#define configKERNEL_INTERRUPT_PRIORITY ( 14 << (8 - configPRIO_BITS) )`
- `#define configMAX_SYSCALL_INTERRUPT_PRIORITY ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY << (8 - configPRIO_BITS) )`
- `#define configASSERT(x) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for( ;; ); }`
- `#define vPortSVCHandler SVC_Handler`
- `#define xPortPendSVHandler PendSV_Handler`

Variables

- `char _end`
- `char _estack`
- `char _Min_Stack_Size`

8.7.1 Detailed Description

FreeRTOS configuration for STM32 targets.

8.8 STM32FreeRTOSConfig.h

[Go to the documentation of this file.](#)

```
00001
00006 #ifndef STM32FREERTOS_CONFIG_H
00007 #define STM32FREERTOS_CONFIG_H
00008
00009 /* Begin custom definitions for STM32 */
00010 /* Define memory allocation implementations to use:
00011  * 1 to 5 for heap_[1-5].c
00012  * -1 for heap_useNewlib_ST.c
00013  * Default -1 see heap.c
00014  */
00015 #define configMEMMANG_HEAP_NB          -1
00016
00017 /* Set to 1 to use default blink hook if configUSE_MALLOCF_FAILED_HOOK is 1 */
00018 #ifndef configUSE_MALLOCF_FAILED_HOOK_BLINK
00019 #define configUSE_MALLOCF_FAILED_HOOK_BLINK 0
00020 #endif
00021 /* Set to 1 to used default blink if configCHECK_FOR_STACK_OVERFLOW is 1 or 2 */
00022 #ifndef configCHECK_FOR_STACK_OVERFLOW_BLINK
00023 #define configCHECK_FOR_STACK_OVERFLOW_BLINK 0
00024 #endif
00025
00026 /* configUSE_CMSIS_RTOS_V2 has to be defined and set to 1 to use CMSIS-RTOSv2 */
00027 #define configUSE_CMSIS_RTOS_V2        1
00028
00029 /* End custom definitions for STM32 */
00030
00031 /* Ensure stdint is only used by the compiler, and not the assembler. */
00032 #if defined(__ICCARM__) || defined(__CC_ARM) || defined(__GNUC__)
00033 #include <stdint.h>
00034 extern uint32_t SystemCoreClock;
00035 #endif
00036
00037 #if defined(configUSE_CMSIS_RTOS_V2) && (configUSE_CMSIS_RTOS_V2 == 1)
00038 /*----- CMSIS-RTOS V2 specific defines -----*/
00039 /* When using CMSIS-RTOSv2 set configSUPPORT_STATIC_ALLOCATION to 1
00040  * is mandatory to avoid compile errors.
00041  * CMSIS-RTOS V2 implmentation requires the following defines
00042  */
00043 /* cmsis_os threads are created using xTaskCreateStatic() API */
00044 #define configSUPPORT_STATIC_ALLOCATION 0
00045 /* CMSIS-RTOSv2 defines 56 levels of priorities. To be able to use them
00046  * all and avoid application misbehavior, configUSE_PORT_OPTIMISED_TASK_SELECTION
00047  * must be set to 0 and configMAX_PRIORITIES to 56
00048  */
00049 /*
00050 #define configMAX_PRIORITIES          (56)
```



```

00051 /*
00052  * When set to 1, configMAX_PRIORITIES can't be more than 32
00053  * which is not suitable for the new CMSIS-RTOS v2 priority range
00054  */
00055 #define configUSE_PORT_OPTIMISED_TASK_SELECTION 0
00056 /* Require to be a value as used in cmsis_os2.c as array size */
00057 #ifndef configMINIMAL_STACK_SIZE
00058 #define configMINIMAL_STACK_SIZE ((uint16_t)128)
00059 #endif
00060
00061 #if !defined(CMSIS_device_header)
00062 /* CMSIS_device_header defined to stm32_def.h by default, which include <device.h> like stm32f1xx.h */
00063 #define CMSIS_device_header "stm32_def.h"
00064 #endif /* CMSIS_device_header */
00065 #else
00066 #define configMAX_PRIORITIES (7)
00067 #endif /* configUSE_CMSIS_RTOS_V2 */
00068
00069 extern char _end; /* Defined in the linker script */
00070 extern char _estack; /* Defined in the linker script */
00071 extern char _Min_Stack_Size; /* Defined in the linker script */
00072 /*
00073  * _Min_Stack_Size is often set to 0x400 in the linker script
00074  * Use it divided by 8 to set minimal stack size of a task to 128 by default.
00075  * End user will have to properly configure those value depending to their needs.
00076  */
00077 #ifndef configMINIMAL_STACK_SIZE
00078 #define configMINIMAL_STACK_SIZE ((uint16_t)((uint32_t)&_Min_Stack_Size/8))
00079 #endif
00080 #ifndef configTOTAL_HEAP_SIZE
00081 #define configTOTAL_HEAP_SIZE ((size_t)((uint32_t)&_estack - (uint32_t)&_Min_Stack_Size - (uint32_t)&_end))
00082 #endif
00083 #ifndef configISR_STACK_SIZE_WORDS
00084 #define configISR_STACK_SIZE_WORDS ((uint32_t)&_Min_Stack_Size/4)
00085 #endif
00086
00087 #define configUSE_PREEMPTION 1
00088 #define configUSE_IDLE_HOOK 1
00089 #define configUSE_TICK_HOOK 1
00090 #define configCPU_CLOCK_HZ (SystemCoreClock)
00091 #define configTICK_RATE_HZ ((TickType_t)1000)
00092 #define configMAX_TASK_NAME_LEN (16)
00093 #define configUSE_TRACE_FACILITY 0
00094 #define configUSE_16_BIT_TICKS 0
00095 #define configIDLE_SHOULD_YIELD 1
00096 #define configUSE_MUTEXES 1
00097 #define configQUEUE_REGISTRY_SIZE 8
00098 #define configCHECK_FOR_STACK_OVERFLOW 0
00099 #define configUSE_RECURSIVE_MUTEXES 0
00100 #define configUSE_MALLOC_FAILED_HOOK 0
00101 #define configUSE_APPLICATION_TASK_TAG 0
00102 #define configUSE_COUNTING_SEMAPHORES 0
00103 #define configGENERATE_RUN_TIME_STATS 0
00104 /*
00105  * If configUSE_NEWLIB_REENTRANT is set to 1 then a newlib reent structure
00106  * will be allocated for each created task.
00107  *
00108  * Note Newlib support has been included by popular demand, but is not used
00109  * by the FreeRTOS maintainers themselves. FreeRTOS is not responsible for
00110  * resulting newlib operation. User must be familiar with newlib and must
00111  * provide system-wide implementations of the necessary stubs.
00112  * Be warned that (at the time of writing) the current newlib design implements
00113  * a system-wide malloc() that must be provided with locks.
00114  */
00115 #define configUSE_NEWLIB_REENTRANT 1
00116
00117 #define configUSE_TIME_SLICING 1
00118
00119 /* Set configENABLE_MPU to 1 to enable MPU support and 0 to disable it. This is
00120 currently used in ARMv8M ports. */
00121 #define configENABLE_MPU 0
00122 /* Set configENABLE_FPU to 1 to enable FPU support and 0 to disable it. This is
00123 currently used in ARMv8M ports. */
00124 #define configENABLE_FPU 1
00125 /* Set configENABLE_TRUSTZONE to 1 enable TrustZone support and 0 to disable it.
00126 This is currently used in ARMv8M ports. */
00127 #define configENABLE_TRUSTZONE 0
00128
00129 /* Co-routine definitions. */
00130 #define configUSE_CO_ROUTINES 0
00131 #define configMAX_CO_ROUTINE_PRIORITIES (2)
00132
00133 /* Software timer definitions. */
00134 #define configUSE_TIMERS 1
00135 #define configTIMER_TASK_PRIORITY (5)
00136 #define configTIMER_QUEUE_LENGTH 10

```

```

00137 #define configTIMER_TASK_STACK_DEPTH      (configMINIMAL_STACK_SIZE )
00138
00139 /* Set the following definitions to 1 to include the API function, or zero
00140 to exclude the API function. */
00141 #define INCLUDE_vTaskPrioritySet           1
00142 #define INCLUDE_uxTaskPriorityGet          1
00143 #define INCLUDE_vTaskDelete                1
00144 #define INCLUDE_vTaskCleanUpResources      1
00145 #define INCLUDE_vTaskSuspend               1
00146 #define INCLUDE_vTaskDelayUntil            1
00147 #define INCLUDE_vTaskDelay                 1
00148 #define INCLUDE_xTaskGetSchedulerState     1
00149 #define INCLUDE_uxTaskGetStackHighWaterMark 1
00150 #define INCLUDE_xTaskGetIdleTaskHandle     1
00151
00152 #if defined(configUSE_CMSIS_RTOS_V2) && (configUSE_CMSIS_RTOS_V2 == 1)
00153 #define INCLUDE_xSemaphoreGetMutexHolder   1
00154 #define INCLUDE_eTaskGetState              1
00155 #define INCLUDE_xTimerPendFunctionCall     1
00156 #define INCLUDE_xTaskGetCurrentTaskHandle 1
00157 #endif
00158
00159 /* Cortex-M specific definitions. */
00160 #ifndef __NVIC_PRIO_BITS
00161 /* __BVIC_PRIO_BITS will be specified when CMSIS is being used. */
00162 #define configPRIO_BITS                     __NVIC_PRIO_BITS
00163 #else
00164 #define configPRIO_BITS                     4          /* 15 priority levels */
00165 #endif
00166
00167 /* The lowest interrupt priority that can be used in a call to a "set priority"
00168 function. */
00169 #define configLIBRARY_LOWEST_INTERRUPT_PRIORITY   0xf
00170
00171 /* The highest interrupt priority that can be used by any interrupt service
00172 routine that makes calls to interrupt safe FreeRTOS API functions. DO NOT CALL
00173 INTERRUPT SAFE FREERTOS API FUNCTIONS FROM ANY INTERRUPT THAT HAS A HIGHER
00174 PRIORITY THAN THIS! (higher priorities are lower numeric values. */
00175 #define configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY 5
00176
00177 /* Interrupt priorities used by the kernel port layer itself. These are generic
00178 to all Cortex-M ports, and do not rely on any particular library functions. */
00179 /* Warning in case of Ethernet, this prio (used by systick) should be higher (lower value) than
00180 ethernet Timer */
00181 #define configKERNEL_INTERRUPT_PRIORITY   ( 14 « (8 - configPRIO_BITS) )
00182
00183 /* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to zero !!!!
00184 See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
00185 #define configMAX_SYSCALL_INTERRUPT_PRIORITY   ( configLIBRARY_MAX_SYSCALL_INTERRUPT_PRIORITY « (8 -
00186 configPRIO_BITS) )
00187
00188 /* Normal assert() semantics without relying on the provision of an assert.h
00189 header file. */
00190 #define configASSERT( x ) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for( ;; ); }
00191
00192 /* Definitions that map the FreeRTOS port interrupt handlers to their CMSIS
00193 standard names. */
00194 #define vPortSVCHandler      SVC_Handler
00195 #define xPortPendSVHandler   PendSV_Handler
00196
00197 /*
00198 * IMPORTANT:
00199 * SysTick_Handler() from stm32duino core is calling weak osSystickHandler().
00200 * Both CMSIS-RTOSv2 and CMSIS-RTOS override osSystickHandler()
00201 * which is calling xPortSysTickHandler(), defined in respective CortexM-x port
00202 */
00203 #define xPortSysTickHandler  SysTick_Handler
00204
00205 #endif /* FREERTOS_CONFIG_DEFAULT_H */

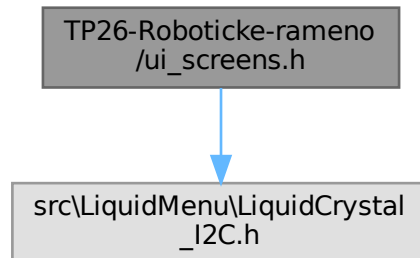
```

8.9 TP26-Roboticke-rameno/ui_screens.h File Reference

User interface display functions for LCD output.

```
#include "src\LiquidMenu\LiquidCrystal_I2C.h"
```

Include dependency graph for ui_screens.h:



Functions

- `void drawXYZ (LiquidCrystal_I2C &lcd, float x, float y, float z)`
Displays end-effector position (X, Y, Z) on LCD screen.
- `void drawRPY (LiquidCrystal_I2C &lcd, float r, float p, float y)`
Displays end-effector orientation (Roll-Pitch-Yaw) on LCD screen.

8.9.1 Detailed Description

User interface display functions for LCD output.

Provides inline utility functions for rendering robot arm state information on a 16×2 character LCD display via I2C connection (LiquidCrystal_I2C).

Functions display:

- End-effector position (X, Y, Z Cartesian coordinates)
- End-effector orientation (Roll-Pitch-Yaw Euler angles)

All values are formatted to 1 decimal place for readability on small displays.

8.9.2 Function Documentation

8.9.2.1 drawRPY()

```
void drawRPY (
    LiquidCrystal_I2C & lcd,
    float r,
    float p,
    float y ) [inline]
```

Displays end-effector orientation (Roll-Pitch-Yaw) on LCD screen.

Parameters

<i>lcd</i>	Reference to LiquidCrystal_I2C display object
<i>r</i>	Roll angle in radians (rotation about X-axis)
<i>p</i>	Pitch angle in radians (rotation about Y-axis)
<i>y</i>	Yaw angle in radians (rotation about Z-axis)

Display layout (16×2 character LCD):

```
Row 0: "R:rrr.r P:ppp.p"
Row 1: "Y:yy.y"
```

All angles are displayed with 1 decimal place precision. Screen is cleared before rendering.

Rotation convention (right-hand rule):

- Roll (R): Rotation about X-axis (shoulder tilt)
- Pitch (P): Rotation about Y-axis (elbow elevation)
- Yaw (Y): Rotation about Z-axis (wrist rotation)

Note

Used during manual control and end-effector orientation monitoring modes. Angles are typically stored internally in radians but displayed here as-is for direct verification against joint sensor values. Consider converting to degrees for user-friendlier display if needed.

8.9.2.2 drawXYZ()

```
void drawXYZ (
    LiquidCrystal_I2C & lcd,
    float x,
    float y,
    float z ) [inline]
```

Displays end-effector position (X, Y, Z) on LCD screen.

Parameters

<i>lcd</i>	Reference to LiquidCrystal_I2C display object
<i>x</i>	End-effector X position in meters
<i>y</i>	End-effector Y position in meters
<i>z</i>	End-effector Z position in meters

Display layout (16×2 character LCD):

```
Row 0: "X:xxx.x Y:yy.y"
Row 1: "Z:zz.z"
```

All coordinates are displayed with 1 decimal place precision. Screen is cleared before rendering.

Note

Used during manual control and end-effector monitoring modes. Typical refresh rate: 100-500 ms (depends on control loop frequency).

8.10 ui_screens.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 #include "src\LiquidMenu\LiquidCrystal_I2C.h"
00004
00038 inline void drawXYZ(LiquidCrystal_I2C& lcd, float x, float y, float z) {
00039     lcd.clear();
00040
00041     lcd.setCursor(0,0);
00042     lcd.print("X:");
00043     lcd.print(x,1);
00044
00045     lcd.setCursor(8,0);
00046     lcd.print("Y:");
00047     lcd.print(y,1);
00048
00049     lcd.setCursor(0,1);
00050     lcd.print("Z:");
00051     lcd.print(z,1);
00052 }
00053
```

```
00080 inline void drawRPY(LiquidCrystal_I2C& lcd, float r, float p, float y) {
00081     lcd.clear();
00082
00083     lcd.setCursor(0,0);
00084     lcd.print("R:");
00085     lcd.print(r,1);
00086
00087     lcd.setCursor(8,0);
00088     lcd.print("P:");
00089     lcd.print(p,1);
00090
00091     lcd.setCursor(0,1);
00092     lcd.print("Y:");
00093     lcd.print(y,1);
00094 }
```


Index

actuator_parameters, [13](#)

d

RotationMatrix, [26](#)

RotationX, [29](#)

RotationY, [31](#)

RotationZ, [33](#)

drawRPY

ui_screens.h, [53](#)

drawXYZ

ui_screens.h, [54](#)

drotX

geometry.h, [37](#)

drotY

geometry.h, [38](#)

drotZ

geometry.h, [38](#), [39](#)

forwardKinematics

Robot::RobotKinematics< DOF >, [19](#)

Robot::RobotKinematicsBase< DOF >, [22](#)

geometry.h

drotX, [37](#)

drotY, [38](#)

drotZ, [38](#), [39](#)

rotX, [39](#)

rotY, [39](#), [40](#)

rotZ, [40](#)

getCurrentEffectorPositionOrientation

Robot::RobotArm< DOF >, [16](#)

getJacobian

Robot::RobotKinematics< DOF >, [20](#)

getJointAngle

Robot::RobotArm< DOF >, [16](#)

getJointAngles

Robot::RobotArm< DOF >, [16](#)

getRotDerivatives

Robot::RobotKinematics< DOF >, [20](#)

goTo

Robot::RobotArm< DOF >, [16](#)

init

RobotServoController, [25](#)

operator()

RotationMatrix, [26](#), [27](#)

RotationX, [29](#)

RotationY, [31](#)

RotationZ, [33](#)

Robot, [11](#)

Robot::JointActuators, [13](#)

writeAngles, [14](#)

Robot::RobotArm< DOF >, [14](#)

getCurrentEffectorPositionOrientation, [16](#)

getJointAngle, [16](#)

getJointAngles, [16](#)

goTo, [16](#)

RobotArm, [16](#)

setJointAngles, [17](#)

Robot::RobotKinematics< DOF >, [17](#)

forwardKinematics, [19](#)

getJacobian, [20](#)

getRotDerivatives, [20](#)

RobotKinematics, [19](#)

SolveIK, [20](#)

Robot::RobotKinematicsBase< DOF >, [21](#)

forwardKinematics, [22](#)

SolveIK, [22](#)

RobotArm

Robot::RobotArm< DOF >, [16](#)

RobotKinematics

Robot::RobotKinematics< DOF >, [19](#)

RobotServoController, [23](#)

init, [25](#)

RobotServoController, [24](#)

writeAngles, [25](#)

RotationMatrix, [25](#)

d, [26](#)

operator(), [26](#), [27](#)

RotationX, [27](#)

d, [29](#)

operator(), [29](#)

RotationY, [29](#)

d, [31](#)

operator(), [31](#)

RotationZ, [31](#)

d, [33](#)

operator(), [33](#)

rotX

geometry.h, [39](#)

rotY

geometry.h, [39](#), [40](#)

rotZ

geometry.h, [40](#)

setJointAngles

Robot::RobotArm< DOF >, [17](#)

SolveIK

Robot::RobotKinematics< DOF >, [20](#)

Robot::RobotKinematicsBase< DOF >, [22](#)

TP26-Roboticke-rameno/geometry.h, [35](#), [41](#)

TP26-Roboticke-rameno/robot.h, [43](#), [44](#)

TP26-Roboticke-rameno/ServoActuator.h, [47](#), [48](#)

TP26-Roboticke-rameno/STM32FreeRTOSConfig.h, [49](#),
[50](#)

TP26-Roboticke-rameno/ui_screens.h, [52](#), [54](#)

TÍMOVÝ PROJEKT, [1](#)

ui_screens.h

- drawRPY, [53](#)
- drawXYZ, [54](#)

writeAngles

- Robot::JointActuators, [14](#)
- RobotServoController, [25](#)